

Student Record and Academic Management System

Data Organization using Python

Student Name: Nikita Dnyaneshwar Udavant

PRN Number: 2127UDSF1031

1. Objective

The objective is to develop a Student Record Management System using Python data structures. The system stores, organizes, updates and retrieves student academic information.

The work follows these steps: problem understanding, algorithm design, flowchart, pseudocode, implementation, testing and documentation.

2. Problem Analysis

Student information such as roll number, name, department, subjects, marks, attendance and contact details has to be stored. The user provides student details or selects an operation. The system gives the required student record or calculated result as output.

3. Data Structure Selection

Data Structure	Use
String	Name, department, address and email ID
List	Student records, subjects, marks and attendance
Tuple	Roll number, registration number and date of birth
Set	Departments, subjects and student clubs
Dictionary	Complete student profile

Example Dictionary

```
student = {  
    "roll_no": 101,  
    "name": "Nikita",  
    "department": "IT",  
    "subjects": ["Python", "Maths"],  
    "marks": [80, 75],  
    "attendance": 90  
}
```

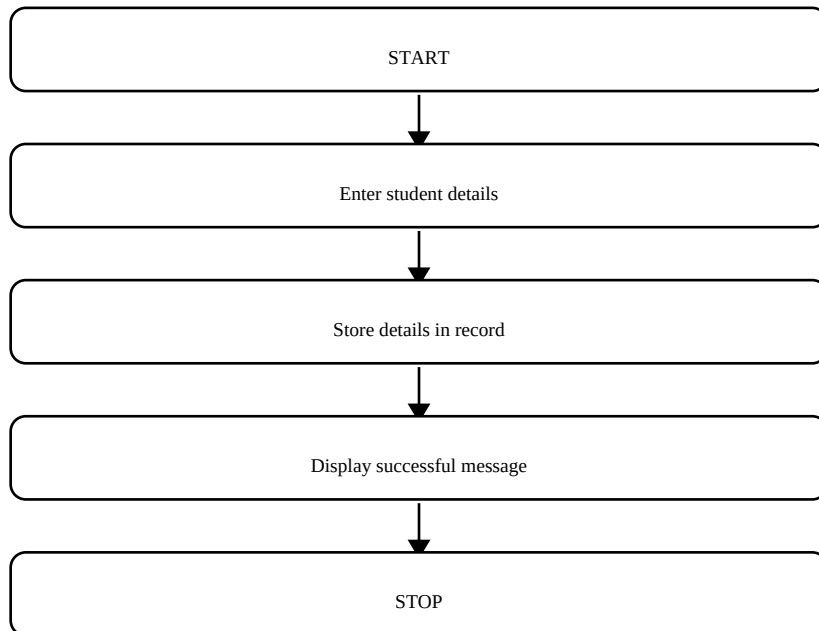
4. Algorithms and Flowcharts

4.1 Add Student

Algorithm:

1. START
2. Enter student details
3. Store details in record
4. Display successful message
5. STOP

Flowchart:

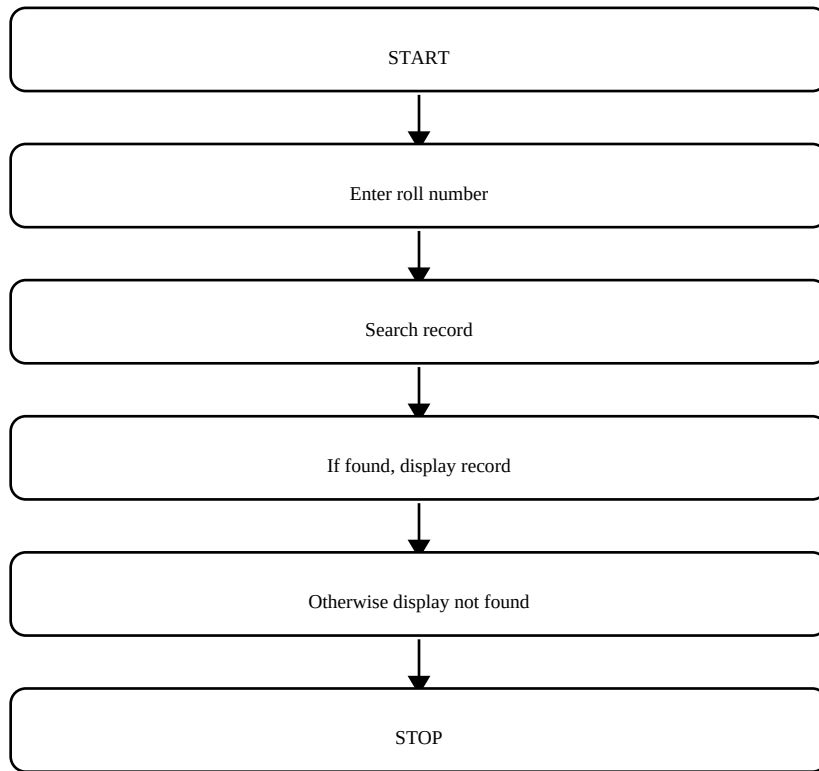


4.2 Search Student

Algorithm:

1. START
2. Enter roll number
3. Search record
4. If found, display record
5. Otherwise display not found
6. STOP

Flowchart:

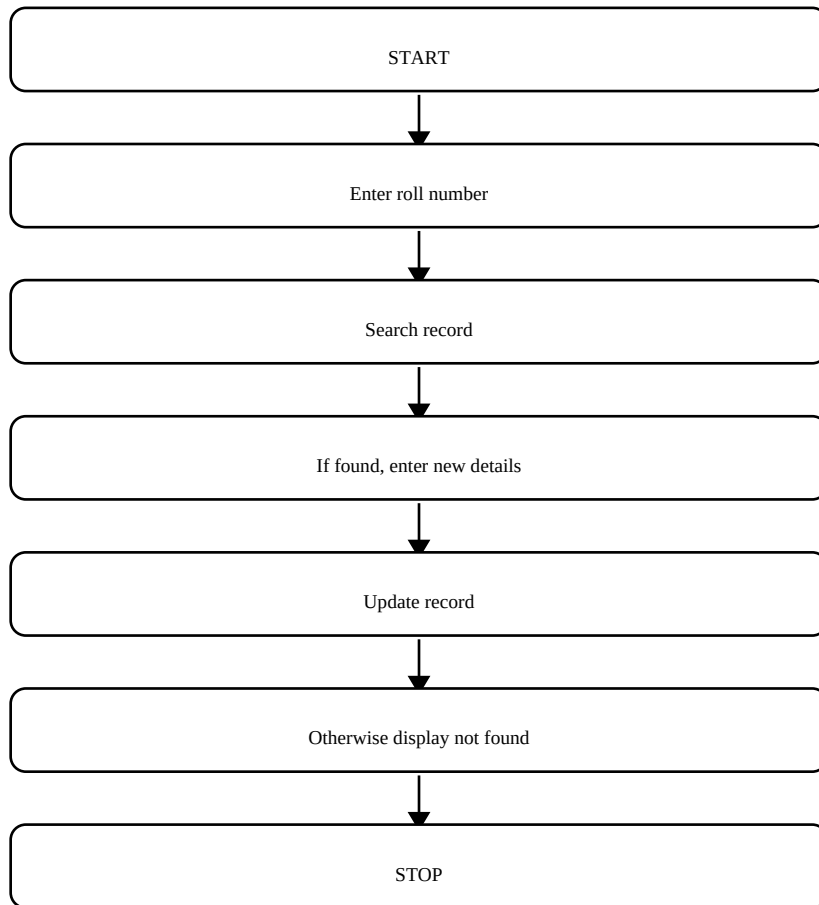


4.3 Update Record

Algorithm:

1. START
2. Enter roll number
3. Search record
4. If found, enter new details
5. Update record
6. Otherwise display not found
7. STOP

Flowchart:

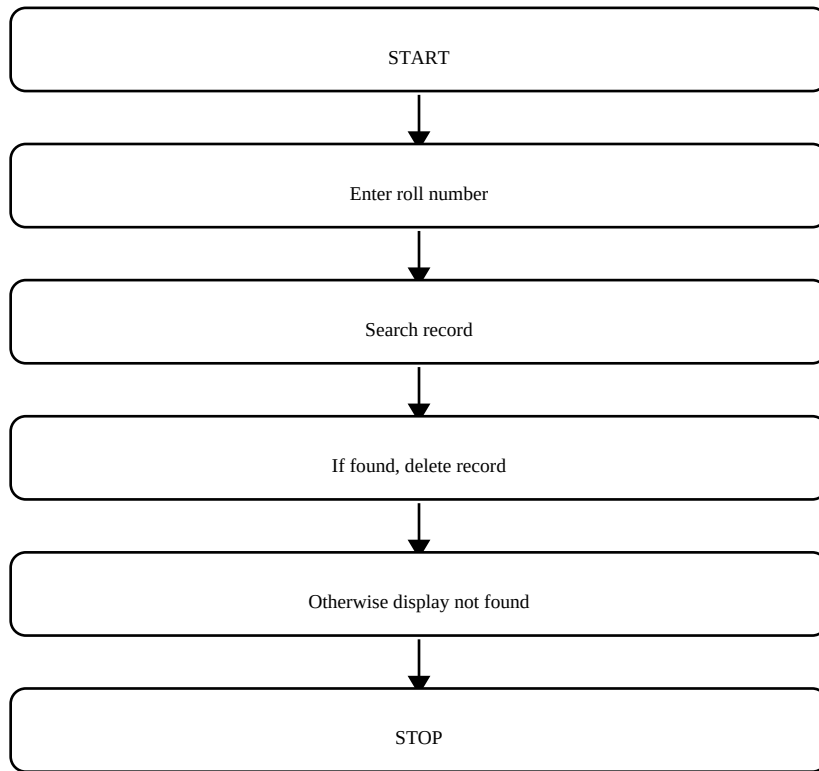


4.4 Delete Record

Algorithm:

1. START
2. Enter roll number
3. Search record
4. If found, delete record
5. Otherwise display not found
6. STOP

Flowchart:

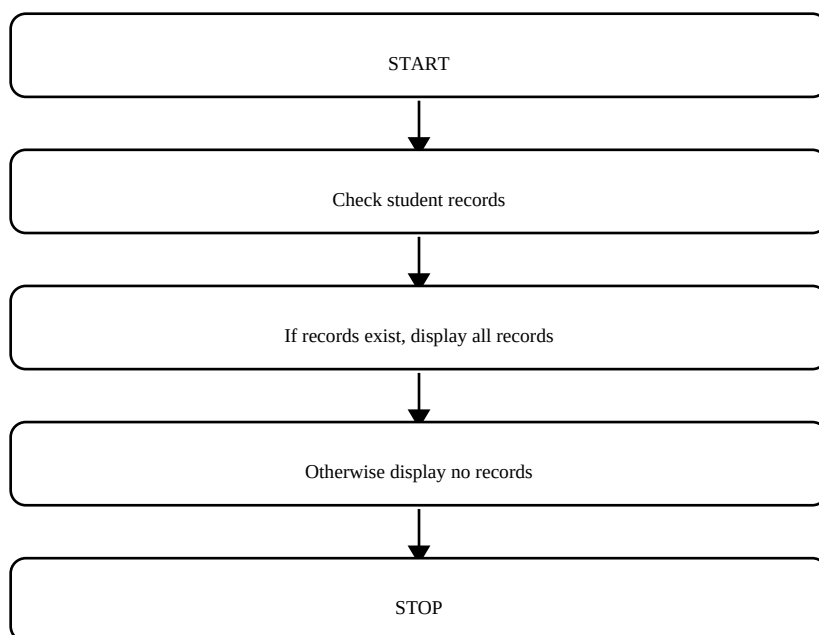


4.5 Display Records

Algorithm:

1. START
2. Check student records
3. If records exist, display all records
4. Otherwise display no records
5. STOP

Flowchart:

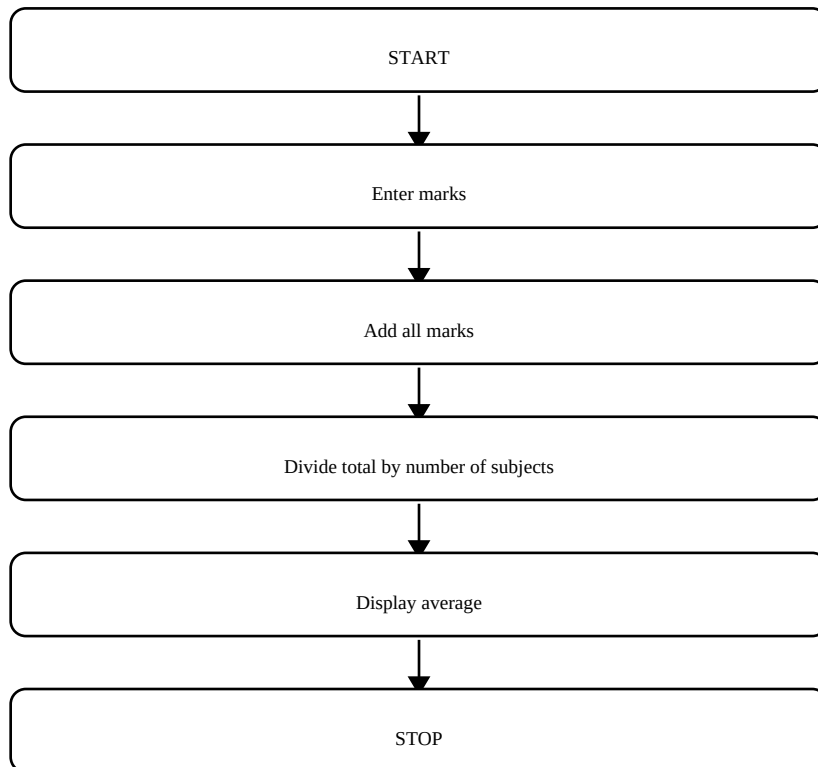


4.6 Calculate Average

Algorithm:

1. START
2. Enter marks
3. Add all marks
4. Divide total by number of subjects
5. Display average
6. STOP

Flowchart:

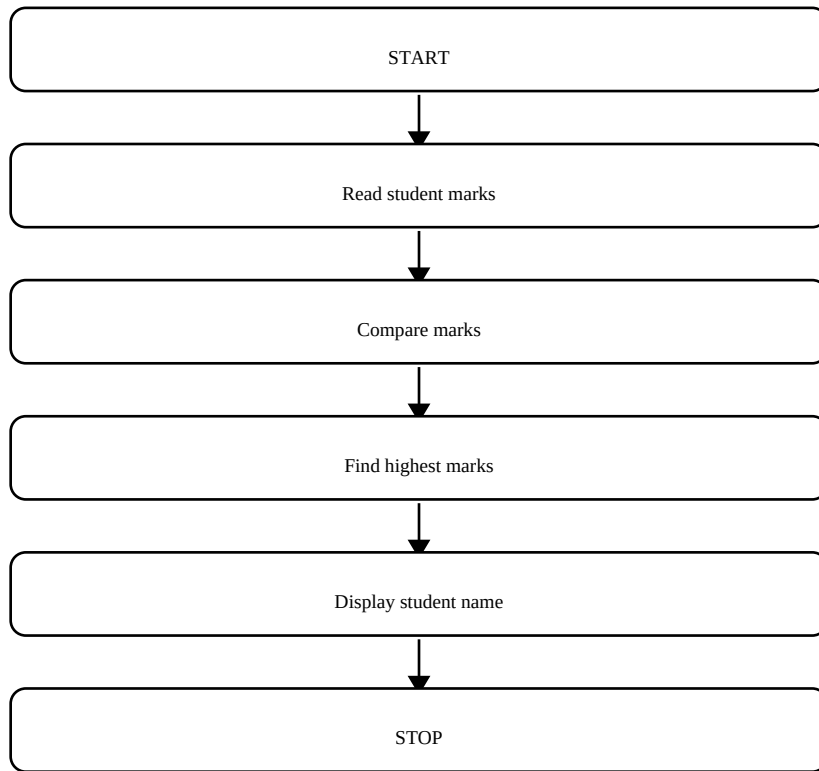


4.7 Find Highest Scorer

Algorithm:

1. START
2. Read student marks
3. Compare marks
4. Find highest marks
5. Display student name
6. STOP

Flowchart:

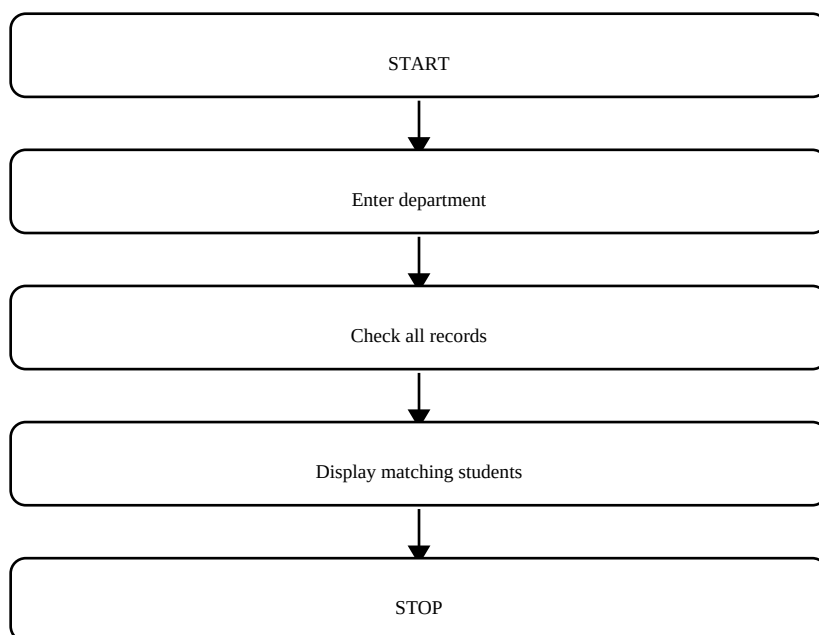


4.8 List by Department

Algorithm:

1. START
2. Enter department
3. Check all records
4. Display matching students
5. STOP

Flowchart:

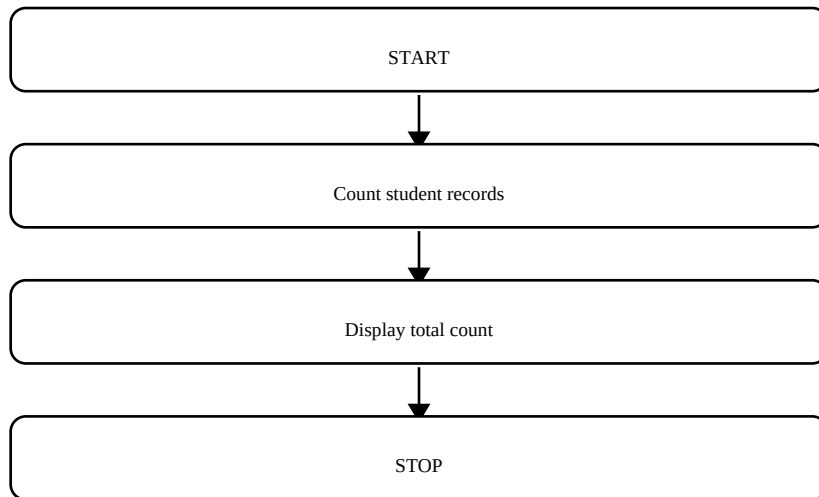


4.9 Count Students

Algorithm:

1. START
2. Count student records
3. Display total count
4. STOP

Flowchart:

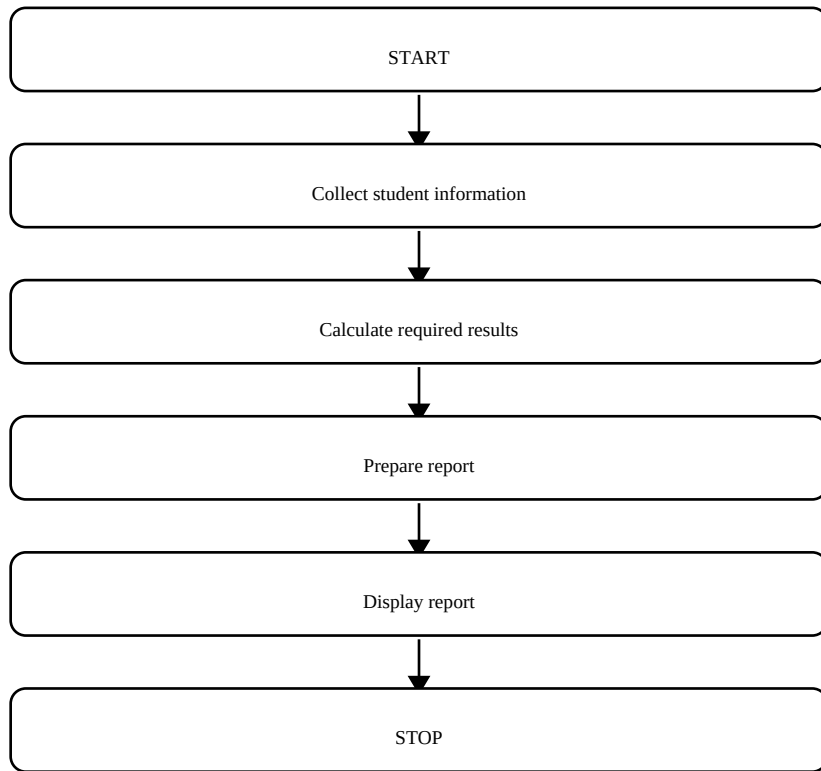


4.10 Generate Reports

Algorithm:

1. START
2. Collect student information
3. Calculate required results
4. Prepare report
5. Display report
6. STOP

Flowchart:



5. Pseudocode

Add Student

```
START
INPUT student details
STORE details
DISPLAY success message
STOP
```

Search Student

```
START
INPUT roll number
SEARCH record
IF found DISPLAY record
ELSE DISPLAY not found
STOP
```

Update Record

```
START
INPUT roll number
SEARCH record
IF found INPUT new details and UPDATE record
ELSE DISPLAY not found
STOP
```

Delete Record

```
START
INPUT roll number
SEARCH record
IF found DELETE record
ELSE DISPLAY not found
STOP
```

Display Records

```
START
IF records are available DISPLAY all records
ELSE DISPLAY no records
STOP
```

Calculate Average

```
START
INPUT marks
TOTAL = sum of marks
AVERAGE = TOTAL / number of subjects
DISPLAY AVERAGE
STOP
```

Find Highest Scorer

```
START
READ all marks
COMPARE marks
STORE highest marks
DISPLAY highest scorer
STOP
```

List by Department

```
START
INPUT department
CHECK all records
DISPLAY matching records
STOP
```

Count Students

```
START
COUNT records
DISPLAY count
STOP
```

Generate Reports

```
START
```

```
COLLECT student data
CALCULATE required results
DISPLAY report
STOP
```

6. Python Implementation

The following simple program uses a list of dictionaries to store student records and demonstrates the main operations.

```
students = []

# Add student
student = {
    "roll_no": 101,
    "name": "Nikita",
    "department": "IT",
    "subjects": ["Python", "Maths"],
    "marks": [80, 75],
    "attendance": 90
}
students.append(student)

# Display records
for s in students:
    print(s)

# Calculate average
marks = student["marks"]
average = sum(marks) / len(marks)
print("Average =", average)

# Count students
print("Total students =", len(students))
```

7. Test Cases

Test	Input / Condition	Expected Result
1	Add valid student	Student record is added
2	Search existing roll number	Student details are displayed
3	Search wrong roll number	Student not found
4	Update existing record	Record is updated
5	Delete existing record	Record is deleted
6	Calculate marks	Correct average is displayed
7	No student records	No records message is displayed
8	Duplicate roll number	Duplicate record should be checked

8. Observations

The program can store student information using different Python data structures. Search, update, delete, average, highest scorer, department listing and counting operations can be performed on the records.

9. Conclusion

The Student Record Management System shows how Python collections can be combined to solve a simple real-world problem. Algorithms, flowcharts, pseudocode, implementation and testing help in developing the system step by step.

10. Project Documentation

The final submission can include this report, source code, test cases and screenshots of the program output.